

The Optimal Search for a Moving Target When the Search Path is Constrained

Author(s): James N. Eagle

Source: *Operations Research*, Sep. - Oct., 1984, Vol. 32, No. 5 (Sep. - Oct., 1984), pp. 1107-1115

Published by: INFORMS

Stable URL: <https://www.jstor.org/stable/170656>

REFERENCES

Linked references are available on JSTOR for this article:

https://www.jstor.org/stable/170656?seq=1&cid=pdf-reference#references_tab_contents

You may need to log in to JSTOR to access the linked references.

JSTOR is a not-for-profit service that helps scholars, researchers, and students discover, use, and build upon a wide range of content in a trusted digital archive. We use information technology and tools to increase productivity and facilitate new forms of scholarship. For more information about JSTOR, please contact support@jstor.org.

Your use of the JSTOR archive indicates your acceptance of the Terms & Conditions of Use, available at <https://about.jstor.org/terms>



INFORMS is collaborating with JSTOR to digitize, preserve and extend access to *Operations Research*

JSTOR

The Optimal Search for a Moving Target When the Search Path Is Constrained

JAMES N. EAGLE

Naval Postgraduate School, Monterey, California

(Received September 1982; accepted September 1983)

A search is conducted for a target moving in discrete time among a finite number of cells according to a known Markov process. The searcher must choose one cell in which to search in each time period. The set of cells available for search depends upon the cell chosen in the last time period. The problem is to find a searcher path, i.e., a sequence of search cells, that maximizes the probability of detecting the target in a fixed number of time periods. We formulate the problem as a partially observable Markov decision process and present a finite time horizon POMDP solution technique which is simpler than the standard linear programming methods.

A DISCRETE time search is conducted for a target moving between a finite set of cells $C = \{1, \dots, N\}$. At the beginning of each time period, one cell is searched. If cell i was searched in the previous time period, the current search cell must be selected from the set $C_i \subseteq C$. If the target is in the selected cell k , it is detected with probability $q_k \in [0, 1]$. If the target is not in the cell searched, it cannot be detected during the current time period. After an unsuccessful search, a target in cell i moves to cell j with probability p_{ij} for the next time period. The transition matrix, $P = [p_{ij}]$, is known to the searcher. The object of the search is to maximize the T -time period probability of detection.

1. PROBLEM MOTIVATION

The problem considered in this paper was motivated by the following antisubmarine warfare (ASW) search problem. A target submarine has been detected by a long-range, but relatively inaccurate, sensor. After the target has been detected, its position is given by a probability distribution. The searcher, a mobile ASW platform, must find the submarine's position for an attack or for closer surveillance. The searcher is provided with the initial target distribution and intelligence on historical target tracks. This track information is in the form of a transition matrix

Subject classification: 567 partially observable Markov decision processes, 751 optimal search for a moving target.

1107

Operations Research
Vol. 32, No. 5, September–October 1984

0030-364X/84/3205-1107 \$01.25
© 1984 Operations Research Society of America

which assigns a probability of the target moving from one discrete cell in the search area to another during one time period. (A slight generalization of the method presented here permits use of a different transition matrix in each time period, allowing greater modeling flexibility.) The searcher's problem is to select that sequence of cells in which to search to maximize the probability of detecting the target by some specified time.

2. BACKGROUND

The moving target problem has received considerable attention, much of it recent. Washburn [1980] and Stone and Kadane [1981] list the important references.

Pollock [1970] solved the problem addressed here for $N = 2$ and $C_1 = C_2 = C$. For any discrete time and space target motion, Brown [1980] found necessary and sufficient conditions for maximizing the probability of detecting the target by the end of a fixed time period when search effort can be infinitely divisible among all search cells and the detection function is exponential. For target motions that are mixtures of Markov processes, he found an efficient algorithm for computing optimal plans. Washburn found necessary conditions for optimal search for targets moving according to a discrete time and space Markov process when detection at one time is independent of detection at any other time period. Washburn also presented an efficient algorithm, similar to Brown's, that yields a sequence of plans with increasing detection probability. When search effort is discrete, Washburn observed that this algorithm may converge to a nonoptimal plan because the space of possible search plans is not convex. Stewart [1980] addressed the search problem considered here by using a discrete version of Brown's algorithm as a "bound" in his branch-and-bound technique. Since the discrete effort version is not guaranteed to produce optimal plans, the "bounds" that Stewart obtained may not be true upper bounds. Thus, his technique is heuristic, but experimental evidence indicates that it is probably a good heuristic.

Smallwood and Sondik [1973] and Monahan [1982] noted that the 2-cell problem solved by Pollock could be modeled as a partially observable Markov decision process (POMDP) and that an N -cell extension was possible. This paper makes that extension and, in addition, allows the set of possible search cells in a given time period to be a function of the search cell selected in the previous time period. This dependence permits us to model situations in which the searcher can travel only a limited distance between time periods. Thus, the search cell in a given time period must be within some specified neighborhood of the search cell in the previous time period.

We also report on a finite time horizon POMDP solution technique which is simpler than the standard linear programming techniques (e.g., Monahan), and which, initial computational experience indicates, is more quickly executed.

3. MATHEMATICAL DEVELOPMENT

As is standard for many problems exploiting a Markov assumption, the solution technique used here is dynamic programming. This method requires that the process being modeled be defined in terms of a sufficient statistic (Bersekas [1976, p. 122]). Following Sondik [1971], Smallwood and Sondik, and Platzman [1980], we use the row vector $(\pi(k), i) \in R^{N+1}$, where $\pi_j(k) = P$ {the target is in cell j at the beginning of time period k , given unsuccessful search in all previous time periods}, and $i \in C$ is the cell searched in the previous time period. If the dependence on k is clear from context, we will write $\pi(k)$ as π . The state space then becomes $\Pi \times C$ where

$$\Pi = \{\pi \in R^N \mid \pi 1 = 1, \pi \geq 0\},$$

and 1 and 0 can be either vectors or scalars. The vector inequality $a \geq b$ means $a_i \geq b_i$ for all i .

Following the dynamic programming convention of labeling "backward in time," we define $V_n(\pi, i)$ to be the maximum obtainable probability of detection with n time periods remaining when the current state vector is (π, i) . Let $T_j(\pi) \in R^N$ be π updated for unsuccessful search in cell j , using Bayes's rule. That is,

$$T_j(\pi) = (1 - q_j \pi_j)^{-1} \pi P_j, \quad (1)$$

where $P_j \in R^{N \times N}$ is P with row j multiplied by $(1 - q_j)$. If $q_j \pi_j = 1$, then the search in the current time period detects the target with certainty, and (1) is not defined. We can now write $V_n(\pi, i)$ in terms of $V_{n-1}(\pi, j)$ as follows:

$$V_n(\pi, i) = \min_{j \in C_i} \{q_j \pi_j + (1 - q_j \pi_j) V_{n-1}(T_j(\pi), j)\}, \quad (2)$$

with $V_0(\pi, i) = 0$.

Equation 2 is the dynamic programming recursion that must be solved in each time period. It looks formidable, primarily because π is real rather than discrete. We will show, however, that $V_n(\pi, i)$ can be expressed in a particularly simple form. Namely,

$$V_n(\pi, i) = \max_{a \in A(n, i)} \pi a, \quad (3)$$

where $A(n, i)$ is a finite collection of N -vectors. The dynamic programming problem then becomes one of constructing $A(n, i)$ from $A(n-1, j)$.

If $C_i = C$ for all i , then the search problem as formulated becomes a standard POMDP and can be solved using the linear programming methods of Sondik, Smallwood and Sondik, or Monahan. Allowing the action selected in the previous time period to constrain the actions available in the present time period requires an augmented state space ($\Pi \times C$ vice Π) and represents a generalization of the standard model. However, as the next theorem shows, the basic form of the POMDP solution remains the same. Specifically, $V_n(\pi, i)$ is piecewise linear and convex.

THEOREM. For $n = 0, \dots, T$, $V_n(\pi, i)$ is piecewise linear and convex in π . That is,

$$V_n(\pi, i) = \max_{a \in A(n, i)} \pi a, \quad (4)$$

where $A(n, i)$ is a finite set of N -vectors.

Proof. We proceed by induction. Equation 4 holds trivially for $n = 0$ and $A(0, i) = 0$. For $n = 1$, it also holds, since from (2),

$$V_1(\pi, i) = \max_{j \in C_i} q_j \pi_j = \max_{a \in A(1, i)} \pi a,$$

where $A(1, i) = \{q_j \xi_j \mid j \in C_i\}$ and $\xi_j \in R^N$ is a column vector with a 1 in the j th place and 0's elsewhere.

Now assume (4) holds in time period $(n - 1)$. From (2),

$$\begin{aligned} V_n(\pi, i) &= \max_{j \in C_i} \{q_j \pi_j + (1 - q_j \pi_j) \max_{a_j \in A(n-1, j)} (T_j(\pi) a_j)\} \\ &= \max_{j \in C_i} \{q_j \pi_j + (1 - q_j \pi_j) \max_{a_j \in A(n-1, j)} (1 - q_j \pi_j)^{-1} \pi P_j a_j\} \quad (5) \\ &= \max_{\substack{j \in C_i \\ a_j \in A(n-1, j)}} \{q_j \pi_j + \pi P_j a_j\} = \max_{a \in A(n, i)} \pi a \end{aligned}$$

where

$$A(n, i) = \{\hat{a} \in R^N \mid \hat{a} = \xi_j q_j + P_j a_j; \quad j \in C_i \text{ and } a_j \in A(n - 1, j)\}. \quad (6)$$

So $V_n(\pi, i)$ is of the proper form and the proof is complete.

For any finite n and $i \in C$, $A(n, i)$ is a finite set. However, using (6) to generate $A(n, i)$ and assuming (for illustration purposes only) that the number of elements in C_i is M for all $i \in C$, the number of vectors in $A(n, i)$ is M times the number in $A(n - 1, i)$. Since there are M vectors in $A(1, i)$, there are apparently M^n vectors in $A(n, i)$, which equals the number of possible search paths for the n -time period problem that begin with cell i . This observation suggests that total enumeration of search paths might be as effective as this procedure.

Fortunately, this is not necessarily the case. Following Smallwood and Sondik, we note that some of the vectors in $A(n, i)$ can be removed

without changing the maximization of (5). We say that $\hat{a} \in A(n, i)$ is *dominated* if for every $\pi \in \Pi$

$$\max_{a \in A(n, i), a \neq \hat{a}} \pi a = \max_{a \in A(n, i)} \pi a. \quad (7)$$

Dominated vectors can be removed from $A(n, i)$ and need not be used in the construction of $A(n + 1, j)$.

Sondik first provided a linear programming technique to identify dominated vectors for the POMDP. Following a slight modification in Monahan, we solve the following linear program to check $\hat{a} \in A(n, i)$ for dominance:

$$\begin{aligned} \min_{\pi, x} \quad & x - \pi \hat{a} \\ \text{subject to: } & x \geq \pi a, \text{ for all } a \in A(n, i) \text{ but } a \neq \hat{a} \quad (8) \\ & \pi \in \Pi. \end{aligned}$$

Whenever the minimal value of $x - \pi \hat{a}$ is nonnegative, $\hat{a}$ is dominated and can be removed from $A(n, i)$. The linear programming solution technique need not necessarily continue to optimality. As soon as the objective function becomes negative, $\hat{a}$ is known not to be dominated. (This method is similar to the branch-and-bound technique of Stewart in that both are enumerative procedures to systematically eliminate search paths that can not be optimal.)

Once the reduced vector sets $A(n, i)$ have been generated for all $i \in C$ and $n = (0, \dots, T)$, the maximum probability of detection and the optimal T -time period search plan can be determined for any initial target distribution π . Assume that before the search begins, the searcher is in cell i , and thus the initial search cell must be in C_i . Then the maximum obtainable T -time period probability of detection is

$$\max_{a \in A(T, i)} \pi a. \quad (9)$$

(If the searcher's starting cell i can be any element in C , (9) is maximized over all $i \in C$ to find the maximum probability of detection.) The cell searched in time period T is that $j \in C_i$ used in (6) to construct the argmax of (9). If cell j is searched in time period T and the target is not detected, then (9) is solved again for time period $T - 1$ with $T_j(\pi)$ replacing π and $A(n - 1, j)$ replacing $A(n, i)$.

Alternatively (and perhaps more simply), one can note that to each $a \in A(T, i)$ is associated, not just the cell searched in time period T , but a series of T cells, built up by the sequential application of (6). When a particular $a \in A(T, i)$ maximizes (9), the sequence of cells associated with the vector a is an optimal search path. As illustrated in the example in Section 6, the optimal search path is not necessarily unique.

Establishing the relationship between each vector a in $A(T, i)$ and a search path allows us to interpret the components of a as probabilities

of detection. Specifically, if the search path associated with any vector a is followed, then the j th component of a is the probability of detecting a target that starts in cell j .

4. THE DUAL DEFINITION OF DOMINANCE AND A GEOMETRIC INTERPRETATION

The linear programming dual of (8) is

$$\begin{aligned} \max \quad & v \\ \text{subject to:} \quad & \sum_{i=1}^k \lambda_i a_i - v \geq \hat{a} \\ & \lambda 1 = 1 \\ & \lambda \geq 0 \end{aligned} \tag{10}$$

where $i = (1, \dots, k)$ indexes all vectors in $A(n, i)$ except $\hat{a}$. The duality theorem of linear programming (Dantzig [1963, p. 125] or Luenburger [1973, p. 72] states that the primal has a finite optimal solution if and only if the dual has a finite optimal solution; and when both problems have optimal solutions, the two optimal objective functions are equal.

We know that $\hat{a} \in A(n, i)$ is dominated when the minimal value of the objective function of (8) is nonnegative. In this case, the duality theorem requires that (10) is feasible and that the optimal value of v is also nonnegative. Thus, from the constraints of (10), there exists a linear combination of elements in $A(n, i)$ except $\hat{a}$ that (in a vector sense) is greater than or equal to $\hat{a}$. And, because of the duality theorem, the implication holds in the other direction as well. That is, if such a linear combination of vectors in $A(n, i)$ exists, then $\hat{a}$ is dominated.

The dual characterization of dominance has a simple geometric interpretation. If B is the convex hull of all vectors in $A(n, i)$ except $\hat{a}$, then $\hat{a}$ is dominated if and only if there is a $b \in B$ such that $b \geq \hat{a}$.

5. ALTERNATIVE SOLUTION TECHNIQUES

The POMDP solution procedure described above requires extensive calculations. To reduce $A(n, i)$ to its minimal size, one must check each $a \in A(n, i)$ for dominance by solving a potentially large linear program. The question naturally arises as to whether a simpler or more quickly executed procedure could be found, even if such a procedure did not necessarily reduce $A(n, i)$ to its minimal size.

What is possibly the simplest such reduction scheme is to compare each a_j and a_k ($a_j \neq a_k$) in $A(n, i)$, and to discard a_j if $a_j \leq a_k$ or $a_k \leq a_j$. The vectors remaining can then be further reduced using linear programming methods, or the possibly larger-than-minimal $A(n, i)$ can be used directly to construct $A(n + 1, j)$ by (6). Both of these procedures

were coded for the IBM 3033 at the Naval Postgraduate School, and, for the relatively small search problems examined, the latter method, using no linear programming at all, generated optimal solutions more quickly. Both methods appeared preferable to using only linear programming methods to check for dominance.

6. AN EXAMPLE PROBLEM

The target and searcher move among the 9 cells shown in Figure 1. In the current time period, the searcher has access to the cell previously

1	2	3
4	5	6
7	8	9

Figure 1. 9-Cell search grid.

searched plus all adjacent cells. Cells are adjacent if they share a common side. Thus,

$$\begin{aligned}
 C &= \{1, 2, \dots, 9\} & C_5 &= \{2, 4, 5, 6, 8\} \\
 C_1 &= \{1, 2, 4\} & C_6 &= \{3, 5, 6, 9\} \\
 C_2 &= \{1, 2, 3, 5\} & C_7 &= \{4, 7, 8\} \\
 C_3 &= \{2, 3, 6\} & C_8 &= \{5, 7, 8, 9\} \\
 C_4 &= \{1, 4, 5, 7\} & C_9 &= \{6, 8, 9\}.
 \end{aligned}$$

During each target transition, the target remains in the previously occupied cell j with probability 0.4 and moves to an adjacent cell with probability $0.6/m_j$, where m_j is the number of cells adjacent to j . So the transition matrix P is

$$\begin{bmatrix}
 0.4 & 0.3 & 0 & 0.3 & 0 & 0 & 0 & 0 & 0 \\
 0.2 & 0.4 & 0.2 & 0 & 0.2 & 0 & 0 & 0 & 0 \\
 0 & 0.3 & 0.4 & 0 & 0 & 0.3 & 0 & 0 & 0 \\
 0.2 & 0 & 0 & 0.4 & 0.2 & 0 & 0.2 & 0 & 0 \\
 0 & 0.15 & 0 & 0.15 & 0.4 & 0.15 & 0 & 0.15 & 0 \\
 0 & 0 & 0.2 & 0 & 0.2 & 0.4 & 0 & 0 & 0.2 \\
 0 & 0 & 0 & 0.3 & 0 & 0 & 0.4 & 0.3 & 0 \\
 0 & 0 & 0 & 0 & 0.2 & 0 & 0.2 & 0.4 & 0.2 \\
 0 & 0 & 0 & 0 & 0 & 0.3 & 0 & 0.3 & 0.4
 \end{bmatrix}$$

The target starts in cell 9, and we assumed that detection is certain if the target is in the cell searched. (That is, $q_i = 1$ for $i = 1, \dots, 9$.)

The optimal search paths for the 10-time period problem and the associated probabilities of detection were computed for each of the 9 possible searcher starting cells. The results appear in Table I. The optimal search path is not unique for any of the searcher starting cells. Note that if cell 9 is the first cell searched, then the probability of

TABLE I
EXAMPLE PROBLEM RESULTS

Searcher Starting Cell	Optimal Search Paths	P_d	No. Possible Search Paths
1	2 3 6 9 8 5 6 5 8 5 2 5 6 9 8 5 6 5 8 5 2 5 8 9 6 5 8 5 6 5 4 5 6 9 8 5 6 5 8 5 4 5 8 9 6 5 8 5 6 5 4 7 8 9 6 5 8 5 6 5	0.7786	492, 803
2	3 6 9 8 8 5 6 5 8 5 5 6 9 8 8 5 6 5 8 5 5 8 9 6 6 5 8 5 6 5	0.8631	696,903
3	3 6 9 8 8 5 6 5 8 5 6 6 9 8 8 5 6 5 8 5	0.8631	492,803
4	5 6 9 8 8 5 6 5 8 5 5 8 9 6 6 5 8 5 6 5 7 8 9 6 6 5 8 5 6 5	0.8631	696,903
5	5 6 9 8 8 5 6 5 8 5 5 8 9 6 6 5 8 5 6 5 6 6 9 8 8 5 6 5 8 5 8 8 9 6 6 5 8 5 6 5	0.8631	985,605
6	— ^a	1	696,903
7	7 8 9 6 6 5 8 5 6 5 8 8 9 6 6 5 8 5 6 5	0.8631	492,803
8	— ^a	1	696,903
9	— ^a	1	492,803

^a All search paths beginning with cell 9 are optimal and result in a P_d of 1.

detection is 1. This outcome is possible if cell 9 is accessible from the searcher starting cell; that is, if the searcher starting cell is 6, 8, or 9.

The calculations required a total of 19.3 CPU minutes on an IBM 3033 and used the simplest reduction method (i.e., no linear programming) coded in FORTRAN H. For a timing comparison, we also wrote a total enumeration routine for an IBM 3033 in FORTRAN H. This program was able to examine approximately 422 10-time period search paths per CPU second and required 152 CPU minutes to identify all the optimal paths for the nontrivial searcher starting cells 1, 2, 3, 4, 5, and 7.

Finally, note that if this problem were to be solved for several initial target distributions π , instead of just one, the POMDP solution method would appear more attractive. Once the vectors sets $A(n, i)$ had been calculated, only about 1.5 CPU seconds were required to find all optimal solutions for any new π . In essence, the dynamic programming calculations required to construct $A(n, i)$ solved the search problem for all possible π . Total enumeration, however, requires that the problem be solved again from the beginning for each new initial distribution of the target.

ACKNOWLEDGMENT

The author is grateful to Lieutenant Doug Guthe, U.S. Navy, for writing the FORTRAN code used to solve the example problem in Section 6. This work was funded by the Foundation Research Program of the Naval Postgraduate School, Monterey, California.

REFERENCES

- BERTSEKAS, D. 1976. *Dynamic Programming and Stochastic Control*. Academic Press, New York.
- BROWN, S. S. 1980. Optimal Search for a Moving Target in Discrete Time and Space. *Opns. Res.* **28**, 1275–1289.
- DANTZIG, G. B. 1963. *Linear Programming and Extensions*. Princeton University Press, Princeton, N.J.
- LUENBERGER, D. G. 1973. *Introduction to Linear and Nonlinear Programming*. Addison-Wesley, Reading, Mass.
- MONAHAN, G. E. 1982. A Survey of Partially Observable Markov Decision Processes: Theory, Models, and Algorithms. *Mgmt. Sci.* **28**, 1–16.
- PLATZMAN, L. K. 1980. Optimal Infinite-Horizon Undiscounted Control of Finite Probabilistic Systems. *Soc. Indust. Appl. Math. J.* **18**, 362–380.
- POLLOCK, S. M. 1970. A Simple Model of Search for a Moving Target. *Opns. Res.* **18**, 883–903.
- SMALLWOOD, R. D., AND E. J. SONDIK. 1973. The Optimal Control of Partially Observable Markov Processes over a Finite Horizon. *Opns. Res.* **21**, 1071–1088.
- SONDIK, E. J. 1971. The Optimal Control of Partially-Observable Markov Processes. Ph.D. thesis, Stanford University, Stanford, Calif.
- STEWART, T. J. 1980. Experience with a Branch-and-Bound Algorithm for Constrained Searcher Motion. In *Search Theory and Applications*, K. B. Haley and L. D. Stone (eds.). Plenum Press, New York.
- STONE, L. D., AND J. B. KADANE. 1981. Optimal Whereabouts Search for a Moving Target. *Opns. Res.* **29**, 1154–1166.
- WASHBURN, A. R. 1980. On a Search for a Moving Target. *Naval Res. Logist. Quart.* **27**, 315–321.